

AN1.1 - AT24 Agent Framework Architecture Audit

Sprint: AT24 AI Agents - Agent Framework Foundation **Stage:** A0 - Existing Architecture Audit (read-only, pre-implementation) **Repo state:** frontend/ on branch fix/quant-lite-stateless-remote-proxy @ 083cae5 **Status:** DRAFT FOR REVIEW - no code written. Implementation is blocked on sign-off of this document (brief sec 30).

0. Executive summary

AT24 already ships a **mock agent scaffold** at /dashboard/agents: an 8-type enum, a Prisma-backed Agent/AgentTask/AgentMemory/AgentActivity quartet, a read-only GET /api/private/agents route, and a client-side services/agents/* "engine" whose planner, executor, tools and memory are all simulated (no real side effects, no run persistence, no evidence, no credit accounting). It is a UI prototype, not a runtime.

Separately - and this is the important part - AT24 has spent ~15 sprints (15D, D2.5, D2.6, D2.7, D2.8) building a **genuinely production-grade, deterministic, evidence-first Market Intelligence stack**: a multi-provider market-data fabric, an Evidence -> Reasoning -> Risk -> Confidence pipeline, a Regime/Hypothesis engine, historical-validation outcome scoring, an immutable per-answer audit trace, and a vendor-independent AI-provider abstraction used *only* for presentation. There is also a canonical deterministic backtest engine (at24-quant-engine) already wired through a Tool-Registry-shaped seam (AlgoTestRun + algo-test.service).

The opportunity: the real Agent Framework should be a thin, governed **orchestration + persistence + governance layer** on top of these existing deterministic engines, exposed to agents through an AT24-owned Tool Registry. Almost none of the "intelligence" needs to be built - it exists. What is missing is: durable run/step/tool-call/evidence persistence, a real tool contract, a supervisor/planner loop that calls real handlers, a credit ledger (today there is only a per-cycle usage counter, unenforced), resource/loop guardrails, autonomy-level gating, and an evaluation hook.

Recommended framing: treat the current services/agents/* + types/agent.ts + data/mock-agents.ts as a throwaway prototype to be replaced (not extended), keep the Agent Prisma model's *shape* as a starting point but add the run-time models, and build the runtime server-side (not in the client bundle, where it lives today).

1. Existing reusable infrastructure

1.1 Persistence / Prisma (frontend/prisma/schema.prisma, 1243 lines, Postgres)

Concern	What exists	Reuse verdict
Agent definition	Agent model (id, userId, type string, name, description, status, provider default "gemini", version, memoryEnabled, tools String[], capabilities String[], goal, priority, lastRun/nextRun, tasksCompleted, successRate, estimatedCost, soft-delete)	Keep & extend. Good bones. Missing: slug, objective vs instructions split, modelPolicy, autonomyLevel, permissionPolicy, creditPolicy, outputSchema, triggerPolicy, knowledgeSources, versioning table. tools/capabilities as bare String[] is too weak for a real tool contract.
Agent versioning	none (version is a free string on the row)	Build AgentVersion (brief sec 22).
Task queue	AgentTask (agentId, userId, title, status string, finishedAt)	Partial. This is a <i>to-do item</i> , not a <i>run</i> . Keep for the "planned steps" idea or fold into AgentRun.steps.

Concern	What exists	Reuse verdict
Run identity	NONE. No AgentRun, no step trace, no tool-call log.	Build AgentRun + AgentStep + AgentToolCall (brief sec 6). This is the single biggest gap.
Memory	AgentMemory (agentId, userId, key, value - flat KV, String value)	Partial. No scope, retention, read/write policy, type (short-term/long-term/user-context/...). Value is String not Json. Extend heavily (brief sec 9).
Activity log	AgentActivity (append-only agentId/userId/message, no updatedAt)	Good <i>pattern</i> (append-only, mirrored by AuditLog, RequestLog, AnalyticsEvent). Too thin for observability - replace with structured AgentStep/trace.
Evidence	NONE at the agent layer. But see IntelligenceAuditTrace + types/evidence.ts + EvidenceItem/EvidenceBundle in the intelligence pipeline - a real, mature evidence/provenance model already exists.	Build AgentEvidence, but its <i>shape</i> should mirror the existing EvidenceItem (type/source/sourceId/timestamp/data/relevance/confidence). So agent evidence and pipeline evidence are one vocabulary.
Evaluation	NONE for agents. But IntelligenceAnalysisRun + IntelligenceAnalysisOutcome + IntelligenceAnalysisOutcomeStatus (pending/validated/invalidated/inconclusive) is a real outcome-evaluation precedent.	Build AgentEvaluation; reuse the enum vocabulary and the "evaluationBasis is always a human-readable reason string" discipline.
Credits / usage	Billing model + config/plan-limits.ts (aiCredits per plan) + EntitlementService + UsageMeteringService + RequestLog. Produces an Entitlements object with aiMessages {used, limit, remaining, atLimit}.	Partial - and critically, NOT enforced. EntitlementService header explicitly says "Enforcement inside the AI Assistant / Knowledge / Market Intelligence pipelines is intentionally NOT wired." There is no credit ledger (no per-action debit rows), only a period-scoped aiMessages counter derived from Message rows + RequestLog counts. See sec 8.
Audit	AuditLog (immutable, no soft-delete, no update path anywhere - actorUserId/action/targetType/targetId/metadata). IntelligenceAuditTrace (immutable per-answer trace).	Reuse AuditLog directly for agent admin actions (create/publish/grant-permission). Model AgentRun's immutability on IntelligenceAuditTrace's "new row, never rewrite" rule.
Ownership	Every model carries denormalized userId (indexed, <i>not</i> FK-constrained to User - established convention, see IntelligenceAnalysisRun comment). Soft-delete deletedAt on mutable models; pure event logs omit it.	Follow exactly.
Auth	lib/auth/protectedRoute.ts - getUserOrNull() / assertRole("admin") / requireUser(). SessionService.getSessionUser() -> { profile: { id, role } }. Supabase-backed.	Reuse directly.

Concern	What exists	Reuse verdict
Cron auth	<code>lib/intelligence/cron-auth.ts</code> - <code>isValidCronSecret(req)</code> , constant-time bearer compare, accepts Vercel's native <code>CRON_SECRET</code> .	Reuse directly for scheduled agent triggers.

1.2 The deterministic intelligence stack (the crown jewels - `services/ai/*`, `services/intelligence/*`)

All of this is **already built, tested (via `scripts/validate-*.ts`), and deterministic**. The Agent Framework should *call* it, never reimplement it (brief sec 16-17, sec 26).

- **Market data fabric** - `services/market-data/*`, `lib/market-data/*`. Multi-provider (MT5/Exness primary, Twelve Data, Binance, Alpha Vantage, Angel One), with `Reliability()` wrapper, freshness policy, cross-provider validation, canonical instrument catalog + live instrument discovery.
- **MarketIntelligencePipelineService** (`services/ai/market-intelligence-pipeline.service.ts`, `MARKET_INTELLIGENCE_PIPELINE_VERSION = "15D.12.0"`) - chains `EvidenceCollector` -> `EvidenceFusion` -> `EvidenceRanking` -> `ReasoningEngine` -> `RiskEngine` -> `ConfidenceEngine` into an immutable `MarketIntelligenceResult` (deep-frozen). **Zero LLM content**.
- **RealTimeIntelligenceService** (`services/intelligence/orchestration/real-time-intelligence.service.ts`) - the single production orchestrator: raw trader question -> `VerifiedRealTimeIntelligenceContext`. Wires query parsing (D2.6.2) -> market data (D2.6.3/4) -> market-state/regime/hypothesis (D2.5.2/3) -> 15D pipeline -> historical validation (D2.5.4) -> envelope (D2.5.5) -> decision context (D2.6.1) -> analysis-run persistence (D2.5.1). **Fully deterministic - no Gemini/Claude/OpenAI call anywhere in it**.
- **Regime / Hypothesis** - `services/intelligence/{market-state, regime, hypothesis}/*`, with `IntelligenceAnalysisRun.hypothesisSnapshot` + `HypothesisOutcomeEvaluatorService` reaching real validated/invalidated verdicts.
- **Evidence & audit** - `types/evidence.ts` (`EvidenceItem`, `EvidenceBundle`), `types/intelligence-audit-trace.ts`, `IntelligenceAuditTrace` model + `AuditTraceService` (create-only). Immutable provenance is a solved problem here.
- **News / events** - `services/ai/news.service.ts`, `economic-calendar.service.ts`, `news-impact.service.ts`, `AlphaVantageNewsProvider`.
- **Microstructure** - `services/microstructure/*`, `services/intelligence/microstructure/*` (order-flow evidence, Binance production integration).
- **Presentation (the ONLY place an LLM is allowed)** - `services/intelligence/chat/ai-presenter-orchestrator.service.ts`, `gemini-intelligence-presenter.service.ts`, `ai-response-integrity.service.ts`, `deterministic-safe-fallback-presenter.service.ts`. The LLM turns a frozen envelope into prose; an integrity check rejects any claim not backed by the envelope; a deterministic fallback presenter exists for when the LLM fails or violates integrity.

1.3 Backtest / Quant engine

- **at24-quant-engine** (sibling package `../at24-quant-engine`, imported as `at24-quant-engine`) - deterministic simulation authority: `runSimulation`, `buildGoldenStrategySpec`, strategy-IR, MQL importer, reality models (spread/slippage/fee/latency), risk-evaluation, reduction. **This is the deterministic execution authority (brief sec 17)**.
- **`services/algo-test/algo-test.service.ts`** + `run-golden-backtest.ts` + `AlgoTestRun` model - the P3.2B production wiring: validate request -> fetch real historical bars (Twelve Data) -> call the engine -> persist bounded metrics/trades/equityCurve. **This is already a Tool-Registry-shaped seam** (typed request -> typed result view, user-owned rows, explicit support surface, `MAX_RANGE_DAYS` guardrail). The Backtest Agent tool wraps *this*, not the engine directly.
- Narrow today: `strategyId="golden"`, `symbol="XAUUSD"`, `timeframe="5m"`. Widening is additive.

1.4 AI provider abstraction (vendor independence - already achieved)

Two parallel abstractions exist (needs consolidation - see sec 2 risks):

- 1. **lib/ai/*** (the clean one): AIProvider interface (name, complete(req)), AIService (constructor-DI, provider + ConversationPort), container.ts composition root, env-driven selectProvider(). Providers: GeminiProvider (real, @google/genai), ClaudeProvider, OpenAIProvider, PlaceholderProvider, GeminiEmbeddingProvider. compliance.ts, disclaimer.ts, response-policy.ts, terminology.ts for guardrails.
- 2. **services/ai/providers/*** (the older client-flavored one): AIProvider type with generate/chat/summarize/analyze/stream/healthCheck, providerRegistry (mock/openai/claude/gemini/deepseek/ollama), provider-factory.ts. The geminiProvider here just fetch("/api/ai").

Neither provider layer supports LLM tool/function-calling. grep for functionDeclarations / tool_use / toolCall across lib/ai and services/ai/providers -> **zero hits**. Implication: the agent runtime's tool *selection* must be AT24-owned (planner/supervisor decides), at least initially - we cannot lean on a provider's native tool-calling. This is *fine* and arguably *better* for governance, but it's a design constraint to state up front.

1.5 API / server conventions

- withContext(handler) wrapper (services/backend/Middleware.ts) - injects RequestContext (requestId, startedAt, path, method), logs start/complete, routes errors through ErrorHandler.
- ApiResponse.success(data, requestId, status, startedAt) / ApiResponse.error({code, message}, requestId, status, startedAt).
- Private routes live under app/api/private/*, guard with getUserOrNull() -> 401 UNAUTHORIZED envelope.
- Client access via typed services/api/*Api.ts classes over ApiClient (TTL cache, retries, invalidate()).
- AGENTS.md: **"This is NOT the Next.js you know"** - read node_modules/next/dist/docs/ before writing route code; APIs/conventions may differ from training data.

1.6 Background / scheduled infrastructure

- **Vercel Cron** (vercel.json): exactly one job today - /api/private/admin/intelligence/evaluate-outcomes daily at 02:00, authed via isValidCronSecret.
- **No general job queue / worker.** WorkflowQueueItem + WorkflowRun exist for the Automations feature but there's no evidence of a running consumer - looks like synchronous execution on trigger.
- Deployment is **Vercel serverless** - long-running agent loops must fit function timeout limits or be broken into resumable steps (see sec 13, sec Risks). Recent commits (faf4dce, 083cae5) show active work routing job stores to os.tmpdir() on Vercel and building "stateless VPS proxy for remote execution" for quant-lite - i.e. the team is *already* dealing with "serverless can't hold state / run long" for backtests.

1.7 Testing

- **No Jest/Vitest.** No *.test.ts / *.spec.ts files. No test script in package.json.
- Testing = **~94 standalone scripts/validate-*.ts files**, each a hand-rolled assert-based harness run via npm run validate:<name> (tsx). Pattern: build deterministic fake inputs -> run the real unmodified service chain -> assert.strict on the output -> print pass/fail counts.
- Implication: the brief's "Testing Requirements" (sec 24) should be delivered as scripts/validate-agent-*.ts files in this exact style, added to package.json, not as a new test-runner dependency (unless we deliberately choose to introduce one - that's a decision for the review).

2. Existing gaps (what the sprint must actually build)

#	Gap	Severity	Notes
G1	No AgentRun / run identity. Nothing persists what an agent did, when, which steps, which tools, what it cost, why it stopped.	Blocker	Brief sec 6, sec 15. Everything else (evidence, evaluation, observability, credits) hangs off this.

#	Gap	Severity	Notes
G2	No real Tool Contract / Tool Registry. <code>tools String[]</code> on <code>Agent</code> ; <code>AgentTools.ts</code> resolves ids against <code>data/mock-agents.ts</code> ; <code>resolveTools</code> returns <code>{id,name,description}</code> with no <code>input_schema</code> , <code>output_schema</code> , <code>permissions</code> , <code>credit_cost</code> , <code>execution_handler</code> .	Blocker	Brief sec 7. The <code>algo-test.service</code> seam shows the <i>shape</i> a real tool handler should have.
G3	Runtime is a client-side mock. <code>services/agents/*</code> runs in the browser bundle, uses module-level <code>let</code> singletons, <code>planTasks</code> = one task per tool + "synthesize", <code>executeTasks</code> = <code>markDone()</code> + push a string. No real planning, no real execution, no LLM call, no persistence of the run.	Blocker	Brief sec 4, sec 6, sec 23 ("UI must be a consumer of the Agent API/runtime"). Runtime must move server-side.
G4	No credit ledger + no enforcement. Only a period <code>aiMessages</code> counter, and <code>EntitlementService</code> explicitly is not called from any pipeline. No pre-flight cost estimate, no per-action debit, no hard stop at limit.	Blocker (safety)	Brief sec 12. <code>estimatedCost Float</code> on <code>Agent</code> is decorative.
G5	No resource/loop guardrails. No <code>max_steps</code> / <code>max_tool_calls</code> / <code>max_runtime</code> / <code>max_credit_cost</code> / <code>max_retries</code> anywhere. <code>AGENT_CONFIG.maxConcurrentTasks</code> = 5 is the only limit and it's client-side and unenforced.	Blocker (safety)	Brief sec 13. Serverless timeout is an <i>implicit</i> ceiling only.
G6	No permission / guardrail contract. No <code>CAN_*</code> capability grants, no default-deny, no <code>LIVE_EXECUTION = DENY</code> . <code>Agent.capabilities String[]</code> is free-text UI copy.	Blocker (safety)	Brief sec 10-11.
G7	No autonomy levels. No LEVEL 0-4 concept.	High	Brief sec 10. Default must be L0/L1; Trading Decision Agent decision-support only.
G8	No agent-layer Evidence contract. Pipeline has one (<code>EvidenceItem</code>); agents can't produce/attach evidence to a run.	High	Brief sec 8. Reuse the pipeline vocabulary.

#	Gap	Severity	Notes
G9	Memory model too thin. Flat key/value:String, no scope/type/retention/policy, no retrieval policy, no permission enforcement.	High	Brief sec 9.
G10	No evaluation hook for agent runs.	Medium	Brief sec 14. <code>IntelligenceAnalysisOutcome</code> is the precedent.
G11	No supervisor/orchestrator abstraction. <code>AgentEngine.runAgentPipeline</code> is the closest thing and it's a 10-line mock.	High	Brief sec 5. Should be runtime architecture, not a user-facing agent type.
G12	Two AI provider abstractions (<code>lib/ai</code> vs <code>services/ai/providers</code>), neither with tool-calling.	Medium	Consolidate on <code>lib/ai/*</code> for the runtime; leave <code>services/ai/providers</code> alone if other code depends on it (don't expand scope).
G13	No write path for agents. <code>GET /api/private/agents</code> only. Agents are created solely by <code>prisma/seed.ts</code> . No <code>create/update/run/cancel</code> endpoints.	High	Brief sec 23. Runtime needs <code>POST /runs</code> , <code>GET /runs/:id</code> , etc.
G14	AgentType enum mismatch. Code enum is <code>market-analyst trading-copilot risk-manager seo-writer news-researcher portfolio-advisor customer-support strategy-generator</code> . Brief's locked 8 are <code>RESEARCH MARKET_INTELLIGENCE STRATEGY_RESEARCH BACKTEST_OPTIMIZATION RISK NEWS_EVENT TRADING_DECISION PORTFOLIO</code> .	Medium	Brief sec 1, sec 5. Needs a registry + migration decision (the seeded rows use the old vocabulary).
G15	Runtime not vendor-independent where it matters. <code>Agent.provider</code> defaults to <code>"gemini"</code> ; <code>AGENT_CONFIG.defaultProvider = "gemini"</code> ; <code>AgentConversation.ts</code> hard-wires the assistant service.	Medium	Brief sec 2. The <i>contract</i> must not name a vendor; <code>modelPolicy</code> should be <code>{ preferred, fallback, allowed[] }</code> .

3. Proposed AT24 Agent Contracts (v1, vendor-neutral)

All contracts are **AT24-owned TypeScript types** under a new `types/agent-framework/` (distinct from the legacy `types/agent.ts`, which the prototype UI keeps using until A15). Versioned via a `contractVersion: "AF-v1"` literal on the root types, following the `M11-license-v1 / licenseSchemaVersion` precedent.

3.1 AgentDefinition (replaces the thin Agent)

```
AgentDefinition
|--- id, slug, version           // slug unique per user; version -> AgentVersion rows
|--- name, description
|--- type                       // AgentType registry key (sec 5)
|--- status                     // draft | active | paused | archived
|--- objective                   // the durable goal ("monitor Gold every 15m...")
|--- instructions                // operator system-prompt text
|--- modelPolicy                 // { preferred: string, fallback: string[], allowed: string[], maxContextT
|--- tools                      // ToolBinding[] - { toolId, config?, creditCeiling? } (NOT bare strings)
|--- knowledgeSources            // KnowledgeSourceRef[] - { kind: "collection"|"document", id, readPolicy
|--- memoryPolicy                // MemoryPolicy (sec 3.5)
|--- triggerPolicy               // { manual: bool, schedule?: cron, events: EventType[] }
|--- permissionPolicy            // PermissionPolicy (sec 3.7) - default-deny
|--- autonomyLevel               // 0..4, default 0
|--- creditPolicy                // { perRunCeiling, perDayCeiling, requireEstimateUnder }
|--- outputSchema                // JSON Schema for the agent's final structured output
\--- createdAt, updatedAt, deletedAt
```

3.2 AgentRun + AgentStep + AgentToolCall (the missing spine)

```

AgentRun
|--- id, agentId, agentVersion, userId
|--- status          // queued | planning | running | awaiting_approval | succeeded
|                   // | failed | timeout | credit_limit | step_limit
|                   // | permission_denied | tool_error | model_error | cancelled
|--- trigger         // manual | schedule | event | supervisor
|--- input           // Json - the goal/params for this run
|--- plan            // Json - supervisor/planner output (ordered intended steps)
|--- output          // Json - validated against AgentDefinition.outputSchema
|--- errorCode, errorMessage
|--- limits          // Json snapshot of the effective ceilings for this run
|--- creditsEstimated, creditsConsumed
|--- startedAt, completedAt
\--- metadata        // Json (pipelineVersion, engineVersions, supervisorVersion...)

AgentStep  (append-only, ordered by index)
|--- id, runId, index, kind          // plan | tool_call | evidence | memory_read
|                                   // | memory_write | evaluation | model_call | output
|--- status, summary
|--- input, output                   // Json
|--- startedAt, completedAt, durationMs
\--- creditsConsumed

AgentToolCall  (append-only)
|--- id, runId, stepId, toolId, toolVersion
|--- input, output                   // Json, each validated against the tool's schemas
|--- status          // ok | invalid_input | tool_error | tool_timeout | permission_denied
|--- permissionChecked // which PermissionKey gated it
|--- creditCost
|--- startedAt, completedAt, durationMs
\--- evidenceIds      // AgentEvidence rows this call produced

```

Immutability: AgentStep / AgentToolCall / AgentEvidence have **no update path and no deletedAt** (mirror AuditLog / AgentActivity). AgentRun gets `updatedAt` only for the status/terminal-field transition; terminal runs are never rewritten (a re-run is a new row).

3.3 Tool contract + registry

[illegible]

```

| // | RISK_ENGINE | PORTFOLIO | EXECUTION
|--- inputSchema, outputSchema // JSON Schema (Zod -> JSON Schema)
|--- requiredPermissions // PermissionKey[]
|--- creditCost // { model: "flat" | "estimated", flat?: number, estimator?: fn }
|--- autonomyFloor // minimum AgentDefinition.autonomyLevel to invoke
|--- handler // (input, RunContext) => Promise<ToolResult> (server-side only)
\--- status // active | deprecated | disabled

```

The registry is a **code-level Map<string, ToolDefinition>** (like providerRegistry / services/agents/AgentRegistry), not a DB table - tools are product code, not user data. Adding a tool = register + ship, no migration.

v1 tool set (wrap existing engines, do not rebuild):

toolId	wraps	permission
market.snapshot	services/market-data shared instance	CAN_READ_MARKET_DATA
market.intelligence	RealTimeIntelligenceService (deterministic)	CAN_READ_MARKET_DATA
indicators.compute	at24-quant-engine indicator-engine	CAN_READ_MARKET_DATA
news.search	services/ai/news.service + AlphaVantageNewsProvider	CAN_READ_NEWS
research.knowledge_search	services/knowledge/KnowledgeRetrieval	CAN_RUN_RESEARCH + source read policy
research.web_search	<i>new thin adapter</i> (only genuinely new capability)	CAN_RUN_RESEARCH
strategy.library_lookup	services/ai/strategy.service	CAN_RUN_RESEARCH
backtest.run	services/algo-test/algo-test.service (already the seam)	CAN_RUN_BACKTEST
risk.evaluate	services/ai/risk engine / at24-quant-engine risk-evaluation	CAN_READ_PORTFOLIO (if portfolio-scoped)
portfolio.read	services/dashboard / paper-trading read models	CAN_READ_PORTFOLIO

EXECUTION category ships **registered but disabled**, autonomyFloor: 3, requiredPermissions: [CAN_CREATE_ORDER, CAN_EXECUTE_ORDER], both denied by default (brief sec 11, sec 19).

3.4 AgentEvidence contract

Mirror the existing pipeline EvidenceItem so it's one vocabulary:

```

AgentEvidence
|--- id, runId, stepId, toolCallId?
|--- type // market_data | news | backtest | strategy_result
| // | research_document | indicator | regime | derived
|--- source, sourceId, timestamp
|--- data // Json - the actual tool result slice cited
|--- relevance, confidence // 0..1
|--- provenance // Json - { provider, pipelineVersion, datasetId, freshness, reliability }
\--- createdAt

```

Agent conclusions in AgentRun.output reference evidenceIds. The **integrity-check pattern** from ai-response-integrity.service.ts (reject any claim not backed by evidence) should be reused for agent final output when an LLM writes the narrative.

3.5 MemoryPolicy + AgentMemory (extended)

```

MemoryPolicy
|--- layers          // subset of: SHORT_TERM | RUN_STATE | LONG_TERM | USER_CONTEXT
|                   // | RESEARCH_MEMORY | STRATEGY_MEMORY | PERFORMANCE_MEMORY
|--- retention       // per-layer: { ttlDays | "run" | "persistent" }
|--- readPolicy      // per-layer: own | agent-type | user-global
\--- writePolicy     // per-layer: allow | deny | approval

AgentMemory (extends today's model)
|--- id, agentId, userId, runId?
|--- layer, scope, key
|--- value           // Json (was String)
|--- retention, expiresAt
|--- createdAt, updatedAt, deletedAt

```

RUN_STATE lives on AgentRun/AgentStep (not this table). An agent may only read/write layers its MemoryPolicy grants - enforced in a MemoryGateway service, never by direct Prisma access from tool handlers.

3.6 Autonomy levels (enum, enforced at the tool gate)

L0 ANALYSIS_ONLY - L1 RECOMMENDATION - L2 PAPER_EXECUTION - L3 USER_APPROVED_ACTION - L4

BOUNDED_AUTONOMOUS_ACTION. Default L0. Trading Decision Agent capped at L1 in v1. A tool with autonomyFloor > agent.autonomyLevel -> permission_denied step, run continues or halts per planner. L2 maps to the existing **paper-trading** service (PaperTradingAccount / PaperPosition) - real math, zero real money, already built. L3/L4 are contract-only in this sprint (no execution wiring).

3.7 PermissionPolicy (default-deny)

```

PermissionKey =
  CAN_READ_MARKET_DATA | CAN_READ_NEWS | CAN_RUN_RESEARCH | CAN_RUN_BACKTEST
  | CAN_READ_PORTFOLIO | CAN_GENERATE_SIGNAL | CAN_CREATE_ORDER
  | CAN_EXECUTE_ORDER | CAN_USE_MEMORY

PermissionPolicy = { granted: PermissionKey[] } // everything not listed = denied

```

Hard invariant in code (not just data): CAN_EXECUTE_ORDER and CAN_CREATE_ORDER are **rejected at the policy-parser layer** unless a server-side AGENT_LIVE_EXECUTION_ENABLED env flag is set AND the user/plan is allowlisted - i.e. LIVE_EXECUTION = DENY is enforced twice (parser + tool gate). Mirror licenseStateMachine.ts's "deterministic, fail-closed" framing.

3.8 CreditPolicy + ledger

```

AgentCreditLedgerEntry (append-only, immutable - pure event log like RequestLog)
|--- id, userId, runId, stepId?, toolCallId?
|--- kind          // run_start | tool_call | model_inference | research_search
|                 // | backtest | optimization | large_context
|--- amount        // credits (positive = debit)
|--- balanceAfter
|--- createdAt

CreditPolicy (on AgentDefinition)
|--- perRunCeiling, perDayCeiling
\--- requireEstimateUnder // refuse to start if pre-flight estimate exceeds this

```

Balance = plan aiCredits (from config/plan-limits.ts) minus period ledger debits.

CreditGateway.reserve(estimate) before a run; .debit(actual) per step; **hard stop** -> run ends credit_limit. This is the first *real* credit ledger in the codebase - the sprint should note that existing aiMessages counting stays as-is (don't refactor billing).

3.9 AgentEvaluation

```

AgentEvaluation
|--- id, runId, task
|--- expectedBehavior, actualBehavior

```

```
|--- toolQuality, evidenceQuality, outputQuality, safety    // 0..1 or enum
|--- latencyMs, creditCost
|--- score          // 0..1 composite
|--- method         // "heuristic" (v1) | "llm_judge" (later) | "human"
\--- createdAt
```

v1 = a **heuristic evaluator** (evidence present? output schema-valid? within limits? no permission violations? no integrity-check failures?). LLM-judge is a later hook, not this sprint.

4. Proposed Prisma changes (additive-only, one migration)

Follow the M8/M11 precedent: **new models only, zero changes to existing tables**, migration generated but **not auto-applied** - applying it needs an explicit go-ahead (the pgvector-reset trap on `prisma migrate dev` is a known hazard in this repo per project memory; use `prisma migrate deploy` / reviewed SQL).

New models:

1. `AgentVersion` - `agentId`, `version`, full definition snapshot `Json`, `createdAt`. (immutable)
2. `AgentRun` - as sec 3.2. Indexes: `[userId]`, `[agentId]`, `[status]`, `[createdAt]`.
3. `AgentStep` - as sec 3.2. Index `[runId, index]`. (append-only)
4. `AgentToolCall` - as sec 3.2. Index `[runId]`, `[toolId]`. (append-only)
5. `AgentEvidence` - as sec 3.4. Index `[runId]`, `[toolCallId]`. (append-only)
6. `AgentEvaluation` - as sec 3.9. Index `[runId]`.
7. `AgentCreditLedgerEntry` - as sec 3.8. Index `[userId, createdAt]`, `[runId]`. (append-only)

Existing models - extend only if low-risk, else leave and supersede:

- `Agent`: add nullable columns `slug`, `objective`, `instructions`, `autonomyLevel Int @default(0)`, `modelPolicy Json?`, `permissionPolicy Json?`, `creditPolicy Json?`, `memoryPolicy Json?`, `triggerPolicy Json?`, `outputSchema Json?`, `toolBindings Json?`. Keep `tools String[]` for the legacy UI until A15. **All nullable, all defaulted** -> no backfill needed, seed rows stay valid.
- `AgentMemory`: add nullable `layer`, `scope`, `runId`, `retention`, `expiresAt`, and a new `valueJson Json?` alongside the existing `value String` (don't change the column type - additive). New code writes `valueJson`; legacy reads `value`.
- `AgentTask`: **leave untouched**. Either keep as the legacy UI's "task" concept or stop writing to it. Do not repurpose.
- `AgentActivity`: **leave untouched**. Superseded by `AgentStep` for new runs; legacy UI keeps reading it.

Enums: `AgentRunStatus`, `AgentStepKind`, `AgentToolCallStatus`, `AgentAutonomyLevel`, `AgentEvidenceType` as real Prisma enums (the schema uses enums freely - `RunStatus`, `WorkflowTrigger`, `IntelligenceAnalysisOutcomeStatus`).

5. Proposed AgentType registry

A **code registry** `types/agent-framework/agent-type-registry.ts`:

```
AGENT_TYPE_REGISTRY: Record<AgentType, AgentTypeSpec>
RESEARCH          -> { defaultTools: [research.*, news.search], autonomyCap: 1, defaultPermissions: [CA
MARKET_INTELLIGENCE -> { defaultTools: [market.intelligence, market.snapshot, indicators.compute, news.se
STRATEGY_RESEARCH  -> { defaultTools: [market.intelligence, strategy.library_lookup, backtest.run, risk.
BACKTEST_OPTIMIZATION -> { defaultTools: [backtest.run, indicators.compute], autonomyCap: 1 }
RISK               -> { defaultTools: [risk.evaluate, portfolio.read, market.snapshot], autonomyCap: 1,
NEWS_EVENT         -> { defaultTools: [news.search], autonomyCap: 1 }
TRADING_DECISION   -> { defaultTools: [market.intelligence, risk.evaluate], autonomyCap: 1 /* decision-s
PORTFOLIO          -> { defaultTools: [portfolio.read, risk.evaluate], autonomyCap: 1 }
```

Adding `EXECUTION`, `COMPLIANCE`, `PORTFOLIO_OPTIMIZER`, `STRATEGY_MONITOR`, `DATA_QUALITY` later = one registry entry + (maybe) tools, no runtime change (brief sec 5).

Migration note for the enum rename (G14): the 3 seeded rows use the old vocabulary. Options for review: (a) map old->new in a one-time script (market-analyst->MARKET_INTELLIGENCE, risk-manager->RISK, strategy-generator->STRATEGY_RESEARCH, etc.), (b) keep type as a free string (as it is today) and only validate against the registry on write. **Recommend (b)** - least invasive, matches the current schema, and the registry does the real work.

The **Supervisor/Orchestrator is NOT an AgentType** - it's runtime architecture (brief sec 5), a services/agent-framework/supervisor/* module.

6. Proposed runtime boundaries



Serverless-fit strategy (brief sec 13 + Vercel constraint): a run is a **resumable state machine**.

AgentRuntime.tick(runId) executes the next 1..N steps within a safe time budget, persists progress, and either completes or re-enqueues. Fast agents (Research/MI on cached data) finish in one request; slow ones (backtest sweeps) span ticks driven by cron or a follow-up call - exactly the pattern the team already built for quant-lite remote execution (083cae5). No new microservice (brief sec 26).

7. Proposed implementation sequence (maps to brief sec 27 A0-A15)

Step	Deliverable	Gate
A0	<i>This document.</i>	G01
A1	types/agent-framework/ contracts (definition, run, step, tool-call, evidence, memory, permission, credit, evaluation) + contractVersion + validate-agent-contracts.ts	G01

Step	Deliverable	Gate
A2	ToolRegistry + ToolDefinition + ToolGateway (schema/permission/credit checks) + v1 tool stubs wrapping real engines + validate-agent-tools.ts	G01
A3	Prisma migration (7 new models + additive Agent/AgentMemory columns), not applied - reviewed SQL ready	G01
A4	AgentRuntime resumable state machine: load -> policy -> plan -> tool loop -> evidence -> persist run/steps -> output. LimitEnforcer. RunTracer. validate-agent-runtime.ts (success/timeout/step-limit/tool-limit/credit-limit/permission-denied)	G02
A5	SupervisorService + PlannerService (deterministic first; LLM-assist behind the lib/ai seam)	G02
A6	EvidenceRecorder + integrity check on final output; validate-agent-evidence.ts	G03
A7	MemoryGateway + extended memory model + policy enforcement; validate-agent-memory.ts	G03
A8	PermissionPolicy parser (fail-closed, double LIVE_EXECUTION=DENY) + autonomy gate in ToolGateway; validate-agent-permissions.ts	G04
A9	CreditGateway + AgentCreditLedgerEntry + pre-flight estimate + hard stop; validate-agent-credits.ts	G04
A10	HeuristicEvaluator (auto-attach to every run) + observability trace assertions	G04
A11	Research Agent end-to-end (research.knowledge_search + research.web_search + news.search -> synthesis -> evidence-backed report)	G05
A12	Market Intelligence Agent (wrap RealTimeIntelligenceService -> regime/trend/momentum/vol/levels/risks/confidence/evidence)	G06
A13	Strategy Research Agent (hypothesis -> strategy spec -> backtest.run -> robustness/OOS -> risk.evaluate -> recommendation, all via existing algo-test + at24-quant-engine)	G06
A14	Regression (npm run validate:* all green) + security review (/security-review) + perf pass on serverless timings	G07

Step	Deliverable	Gate
A15	/dashboard/agents UI rebuilt as a consumer of the runtime API (Agent Builder, run trace viewer, evidence panel)	post-G07

8. Credit integration - detail

- **Source of truth for balance:** `PLAN_LIMITS[planId].aiCredits` (already exists) minus sum of `AgentCreditLedgerEntry.amount` for the current billing period (period boundaries from `EntitlementService / Subscription.currentPeriodStart/End`).
- **Pre-flight:** `PlannerService` produces an intended `ToolIntent[]`; `CreditGateway.estimate(intents)` sums `ToolDefinition.creditCost` (flat or estimator). If `> creditPolicy.requireEstimateUnder` -> refuse start (`credit_limit`, never begin). If `estimate <= remaining` -> `reserve()`.
- **Per step:** `ToolGateway` calls `CreditGateway.debit(actual)` after each tool call and each `model_inference`; writes a ledger row; updates `AgentRun.creditsConsumed`.
- **Hard stop:** any debit that would cross `min(remaining, creditPolicy.perRunCeiling, perDayCeiling)` -> run terminates `credit_limit`, partial output + trace preserved.
- **No silent unlimited consumption** (brief sec 12): the runtime cannot invoke a tool without a successful `CreditGateway` check; there is no bypass path.
- **Explicitly out of scope:** final pricing numbers (brief sec 12 - "Do not hardcode final pricing"); refactoring the existing `aiMessages/RequestLog` billing (leave it).

9. Permission / guardrail model - detail

- `PermissionPolicy.granted` is an allowlist; the parser **rejects** `CAN_CREATE_ORDER / CAN_EXECUTE_ORDER` unless `process.env.AGENT_LIVE_EXECUTION_ENABLED === "true"` **and** the requesting user is on an execution allowlist - otherwise the definition fails validation at write time (can't even be saved with those grants).
- `ToolGateway.invoke()` checks, in order: (1) tool status `=== active`; (2) every tool `requiredPermissions[i]` in `agent.permissionPolicy.granted`; (3) `agent.autonomyLevel >= tool.autonomyFloor`; (4) `CreditGateway` ok; (5) `LimitEnforcer` ok (tool-call count). Any failure -> typed `AgentToolCall` row with the failing reason, no handler invocation.
- Risk Agent independence (brief sec 18): RISK type tools are never gated by a Trading Decision Agent's output; the runtime has no "decision overrides risk" path - a RISK run is its own run.
- Admin actions on agents (create / publish / grant permission / enable execution) -> `AuditLog` rows (reuse the existing immutable model).

10. Observability - detail

Every run produces a linear trace already implied by `AgentStep`: `RUN START -> PLAN -> (STEP -> TOOL CALL -> TOOL RESULT -> EVIDENCE -> MEMORY R/W)* -> EVALUATION -> OUTPUT -> RUN END`. `RunTracer` writes each as an `AgentStep` row **and** emits a structured `logger.info` line (`services/backend/Logger.ts`) with `runId` correlation. `GET /runs/:id/trace` returns the reconstructed tree. A developer can answer "what did it do / which tools / what evidence / how much did it cost / why this result" purely from `AgentRun + AgentStep + AgentToolCall + AgentEvidence + AgentEvaluation` - no log spelunking required.

11. Risks

Risk	Impact	Mitigation
Serverless timeouts on multi-step runs (Vercel function limit).	Runs die mid-way.	Resumable <code>tick()</code> state machine (sec 6); cron-driven continuation; the team already solved this shape for <code>quant-lite</code> . Keep A11/A12 agents single-tick on cached data.
No test runner - 94 ad-hoc validate scripts.	Agent safety paths under-tested / slow CI.	Match the house style (<code>scripts/validate-agent-*.ts</code>), wire into <code>package.json</code> . Decide at review whether to introduce Vitest (scope risk).
Two provider abstractions (<code>lib/ai</code> vs <code>services/ai/providers</code>).	Confusion, drift.	Runtime uses <code>lib/ai/*</code> only. Don't touch <code>services/ai/providers</code> (other code may depend on it) - non-goal to unify this sprint.
No LLM tool-calling in either provider.	Planner can't delegate tool selection to the model.	AT24-owned planner/supervisor does selection (better for governance anyway). Provider tool-calling can be added later without contract change.
prisma migrate dev pgvector reset trap (documented in project memory).	Local DB wipe.	Generate migration, hand-review SQL, apply via <code>migrate deploy</code> ; never <code>migrate dev</code> against a DB with the <code>pgvector</code> extension.
Enum vocabulary mismatch (G14) + seeded rows.	Broken reads if enum enforced.	Keep <code>type</code> as validated free string (recommend), or one-time map script.
Scope creep - brief is enormous.	Sprint never lands.	Hard gate system (G01-G07); A11-A13 are the only real agents; A15 UI last; explicit non-goals below.
Credit ledger is new financial-adjacent bookkeeping.	Incorrect debits erode trust.	Append-only immutable ledger (no update path), <code>balanceAfter</code> recorded per row, reconciled against period start; heavy <code>validate-agent-credits.ts</code> coverage.
Client bundle bloat / secrets - current runtime is client-side.	Server engines/keys must never reach the browser.	Runtime is <code>services/agent-framework/*</code> , server-only; client keeps a typed <code>AgentsApi</code> .
Autonomy misconfiguration -> real orders.	Catastrophic.	<code>LIVE_EXECUTION = DENY</code> enforced twice (parser + gate) + env flag + user allowlist; <code>EXECUTION</code> tools ship disabled; L3/L4 contract-only in v1.

12. Explicit non-goals (this sprint)

Restating brief sec 26, made concrete for AT24:

- No** new backtest/simulation engine - `at24-quant-engine` + `algo-test.service` is the authority; agents call it as a tool.
- No** re-implementation of the Market Intelligence pipeline - `RealTimeIntelligenceService` is called as a tool.
- No** autonomous live trading; **no** `EXECUTION` tool wiring; L3/L4 are contract-only.
- No** merging of Automations (`Automation / Workflow*` models) and Agents - they stay separate IA (brief sec 3).
- No** agent marketplace, **no** 20 agents - exactly 3 real agents (Research, Market Intelligence, Strategy Research).
- No** generic chatbot rebadged as an agent (`AgentConversation.ts` is not the model).

- **No** vendor-specific runtime - no OpenAI Agents SDK, no LangGraph, no Gemini-specific control flow; `lib/ai` stays the only LLM seam.
- **No** changes to billing internals (`aiMessages` / `RequestLog` / `EntitlementService`) beyond *adding* the new agent credit ledger.
- **No** changes to unrelated locked areas (native chart, marketplace, licensing, paper-trading internals, homepage, publishing).
- **No** new test framework unless explicitly approved at review.
- **No** UI polish before G07; `/dashboard/agents` rebuild is A15 only.
- **No** microservices; runtime is in-process, serverless-resumable.

13. Open questions for review (blockers on proceeding to A1)

1. **Enum (G14):** validated free-string `type` (recommended) vs. hard Prisma enum + migration script for the 3 seeded rows?
2. **Test runner:** stay with `scripts/validate-agent-*.ts` (house style) or introduce Vitest for the framework only?
3. **Prototype disposition:** confirm `services/agents/*`, `types/agent.ts`, `data/mock-agents.ts` are treated as replaceable (new code under `services/agent-framework/` + `types/agent-framework/`), with the legacy UI left running on the old modules until A15.
4. **Migration application:** confirm the migration is generated + SQL-reviewed this sprint but applied only on explicit go-ahead (M8/M11 precedent).
5. **research.web_search:** is a new outbound web-search adapter in scope for A11, or should the Research Agent be knowledge-base + news only in v1?
6. **Run execution model:** synchronous `POST /runs` for fast agents + `202 + poll` for slow - acceptable, or is a first-class queue/worker wanted now (bigger scope)?
7. **Credit numbers:** confirm no pricing decisions this sprint - ceilings come from `PLAN_LIMITS.aiCredits`, per-tool costs are placeholder constants flagged for a later pricing pass.

End AN1.1. No implementation until the open questions above are resolved and these findings are confirmed consistent with the codebase.